CHAPTER 5

Testing Models

A standard Rails application uses a design pattern called *MVC*, which stands for model-view-controller. Each of the three sections in the MVC pattern is a separate layer of code, which has its own responsibilities and communicates with the other layers as infrequently as possible. In Rails, the model layer contains both business logic and persistence logic, with the persistence logic being handled by ActiveRecord. Typically, all of your ActiveRecord objects will be part of the model layer, but not everything in the model layer is necessarily an ActiveRecord object. The model layer can include various services, value objects, or other classes that encapsulate logic and use ActiveRecord objects for storage.

I'll start the tour of testing the Rails stack with the model layer because model tests have the fewest dependencies on Rails-specific features and are often the easiest place to start testing your application. Standard Rails model tests are very nearly vanilla RSpec. Features specific to Rails include a few new matchers and the ability to set up initial data.

I'll also talk about testing ActiveRecord features such as associations and models. And I'll talk about separating logic from persistence and why that can be a valuable practice for both testing and application development.

What Can You Do in a Model Test?

An RSpec file in the `spec/models` directory is automatically of `type: :model`, which gives you … not a whole lot of new behavior, actually. An add-on gem called rspec-activemodel-mocks,[1] which is maintained by the RSpec core team, includes some mock-object tools specific to use with ActiveModel.

1. https://github.com/rspec/rspec-activemodel-mocks